



МИНОБРНАУКИ РОССИИ  
Федеральное государственное бюджетное образовательное учреждение  
высшего образования  
«МИРЭА – Российский технологический университет»  
РТУ МИРЭА

---

---

Институт информационных технологий (ИИТ)  
Кафедра прикладной математики (ПМ)

**ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ**  
по дисциплине «Технологии и инструментарий анализа больших данных»

**Практическая работа № 6**

Студент группы *ИКБО-06-22, Ляхов Т. А.*

\_\_\_\_\_  
(подпись)

Проверил *Старший преподаватель,  
Приходько Н.А.*

\_\_\_\_\_  
(подпись)

Отчет представлен «\_\_» \_\_\_\_\_ 2025 г.

Москва 2025 г.

# 1. Теоретическое введение

Кластеризация представляет собой один из ключевых методов обучения без учителя в машинном обучении. В отличие от задач классификации, где данные содержат метки, кластеризация работает с неразмеченными данными, цель которых — выявление внутренней структуры и группировка объектов на основе их схожести. Это позволяет исследователям и аналитикам получать ценную информацию о данных даже в отсутствие заранее известных категорий.

Основная идея кластеризации заключается в разделении множества объектов на группы, называемые кластерами, таким образом, чтобы объекты внутри одного кластера были более похожи друг на друга, чем на объекты из других кластеров. Для оценки сходства между объектами используется метрика расстояния, чаще всего евклидово расстояние, хотя выбор метрики может варьироваться в зависимости от специфики задачи. Кластеризация находит применение в разнообразных областях, таких как сегментация клиентов, обнаружение аномалий, анализ данных в биологии, сжатие информации и многое другое.

Среди множества алгоритмов кластеризации можно выделить три наиболее популярных и широко используемых метода. Алгоритм k-средних (k-means) является, пожалуй, самым известным и простым для понимания. Он основан на итеративном уточнении положений центроидов — центров кластеров — с целью минимизации внутрикластерной дисперсии. Однако его существенным ограничением является необходимость заранее задавать количество кластеров, что не всегда очевидно. Для решения этой проблемы применяются такие методы, как "правило локтя" и оценка с использованием коэффициента силуэта, который позволяет более объективно определить оптимальное число групп.

Иерархическая кластеризация предлагает иной подход, строя древовидную структуру (дендрограмму), которая отражает вложенность

кластеров и взаимосвязи между объектами. Этот метод особенно полезен, когда важна не только группировка, но и визуализация иерархии сходства. Алгоритм может быть агломеративным, начиная с отдельных объектов и последовательно объединяя их, или дивизионным, начиная с общего кластера и разделяя его на более мелкие.

Алгоритм DBSCAN (пространственная кластеризация на основе плотности) отличается от предыдущих тем, что он не требует задания количества кластеров и способен обнаруживать кластеры произвольной формы, а также идентифицировать выбросы (шум). Он группирует точки, расположенные в областях с высокой плотностью, и помечает изолированные точки как шумовые. Это делает DBSCAN особенно эффективным для работы с данными, содержащими нерегулярные кластеры и аномалии, хотя его производительность может снижаться на больших объемах данных.

В данной работе будут рассмотрены и применены на практике эти три основных алгоритма кластеризации. Целью является не только освоение их теоретических основ, но и получение навыков выбора и настройки алгоритмов, оценки их результатов и визуализации полученных кластеров. Практическая часть включает работу с реальными данными, их предобработку, кластеризацию с использованием k-means, иерархического метода и DBSCAN, а также интерпретацию и сравнение результатов.

## 2. Постановка задачи

Условие:

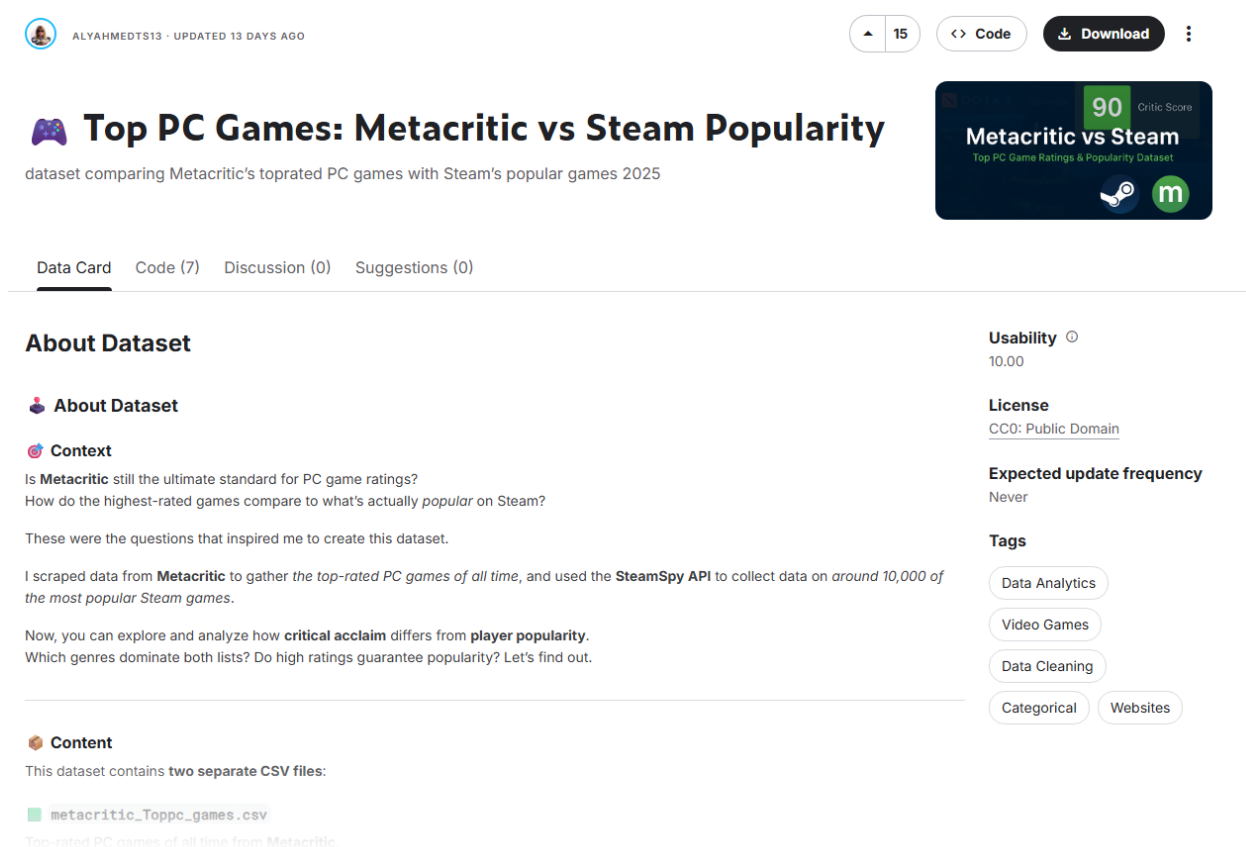
1. *Найти данные для кластеризации. Данные в группе не должны повторяться. Если признаки в данных имеют очень сильно разные масштабы, то необходимо данные предварительно нормализовать.*
2. *Провести кластеризацию данных с помощью алгоритма  $k$ -means. Использовать «правило локтя» и коэффициент силуэта для поиска оптимального количества кластеров.*
3. *Провести кластеризацию данных с помощью алгоритма иерархической кластеризации.*
4. *Провести кластеризацию данных с помощью алгоритма DBSCAN.*
5. *Визуализировать кластеризованные данные с помощью  $t$ -SNE или UMAP, если необходимо. Если данные трехмерные, то можно использовать трехмерный точечный график.*
6. *Оформить отчет о проделанной работе. Сделать выводы.*

### 3. Выполнение практической работы

1. *Найти данные для кластеризации. Данные в группе не должны повторяться. Если признаки в данных имеют очень сильно разные масштабы, то необходимо данные предварительно нормализовать.*

Был найден и скачан датасет «Top PC Games: Metacritic vs Steam Popularity» от AlyAhmedTS13. Этот файл содержит данные, взятые с веб-сайта Metacritic, на котором перечислены самые популярные компьютерные игры всех времен.

Он содержит такие сведения, как названия игр, оценки критиков и пользователей, даты выхода, разработчики, издатели, жанры и количество рецензий. Файл содержит около 10.000 уникальных записей.



ALYAHMEDTS13 · UPDATED 13 DAYS AGO

15 Code Download

## Top PC Games: Metacritic vs Steam Popularity

dataset comparing Metacritic's top-rated PC games with Steam's popular games 2025

90 Critic Score

Metacritic vs Steam  
Top PC Game Ratings & Popularity Dataset

Data Card Code (7) Discussion (0) Suggestions (0)

### About Dataset

**About Dataset**

**Context**

Is **Metacritic** still the ultimate standard for PC game ratings?  
How do the highest-rated games compare to what's actually *popular* on Steam?

These were the questions that inspired me to create this dataset.

I scraped data from **Metacritic** to gather the *top-rated PC games of all time*, and used the **SteamSpy API** to collect data on *around 10,000 of the most popular Steam games*.

Now, you can explore and analyze how **critical acclaim** differs from **player popularity**.  
Which genres dominate both lists? Do high ratings guarantee popularity? Let's find out.

**Content**

This dataset contains **two separate CSV files**:

- `metacritic_Toppc_games.csv`  
Top-rated PC games of all time from Metacritic.

**Usability** 10.00

**License**  
CC0: Public Domain

**Expected update frequency**  
Never

**Tags**

- Data Analytics
- Video Games
- Data Cleaning
- Categorical
- Websites

Рисунок 1 – AlyAhmedTS13 “Top PC Games: Metacritic vs Steam Popularity”

Проведем предобработку данных.

Данные не содержат пропущенных значений, все признаки числовые, что позволило ограничиться минимальной предобработкой: кодированием целевой переменной и стандартизацией признаков для обеспечения

корректной работы моделей SVM и KNN.

После загрузки была выполнена предобработка данных. Код реализации представлен в таблице 1.

Листинг 1 – Исходный код задания 1

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler

# Загрузка набора данных
data = pd.read_csv('dataset/steam_spy_data.csv')

# Функция для парсинга диапазона владельцев и вычисления среднего
def parse_owners(owners_str):
    if '..' in owners_str:
        parts = owners_str.split('..')
        low = int(parts[0].replace(',', ' ').strip())
        high = int(parts[1].replace(',', ' ').strip())
        return (low + high) / 2
    else:
        return int(owners_str.replace(',', ' ').strip())

# Применение к столбцу владельцев
data['owners_avg'] = data['owners'].apply(parse_owners)

# Выбор числовых столбцов для кластеризации
numerical_cols = ['positive', 'negative', 'userscore', 'owners_avg',
                  'average_forever', 'average_2weeks',
                  'median_forever', 'median_2weeks', 'price',
                  'initialprice', 'discount', 'ccu']

# Удаление строк с пропущенными значениями в выбранных столбцах
data_clean = data[numerical_cols].dropna()

# Нормализация данных
scaler = StandardScaler()
data_normalized = scaler.fit_transform(data_clean)

# Сохранение нормализованных данных для использования в других задачах
np.savetxt('prepared_data.csv', data_normalized, delimiter=',')
print("Подготовка данных и нормализация завершена. Сохранено в prepared_data.csv")
```

2. *Провести кластеризацию данных с помощью алгоритма k-means. Использовать «правило локтя» и коэффициент силуэта для поиска оптимального количества кластеров.*

Алгоритм k-means был применён для количества кластеров от 2 до 10.

С использованием «правила локтя» было обнаружено постепенное замедление снижения инерции, однако нечёткий «излом» затруднял однозначный выбор.

Коэффициент силуэта показал чёткий максимум при  $k = 4$ , что было

принято за оптимальное число кластеров.

Это означает, что при четырёх кластерах объекты наиболее плотно сгруппированы внутри кластеров и максимально удалены от соседних кластеров, что подтверждает обоснованность выбора.

Код реализации представлен в таблице 2. Результат выполнения программы представлен на рисунке 1.

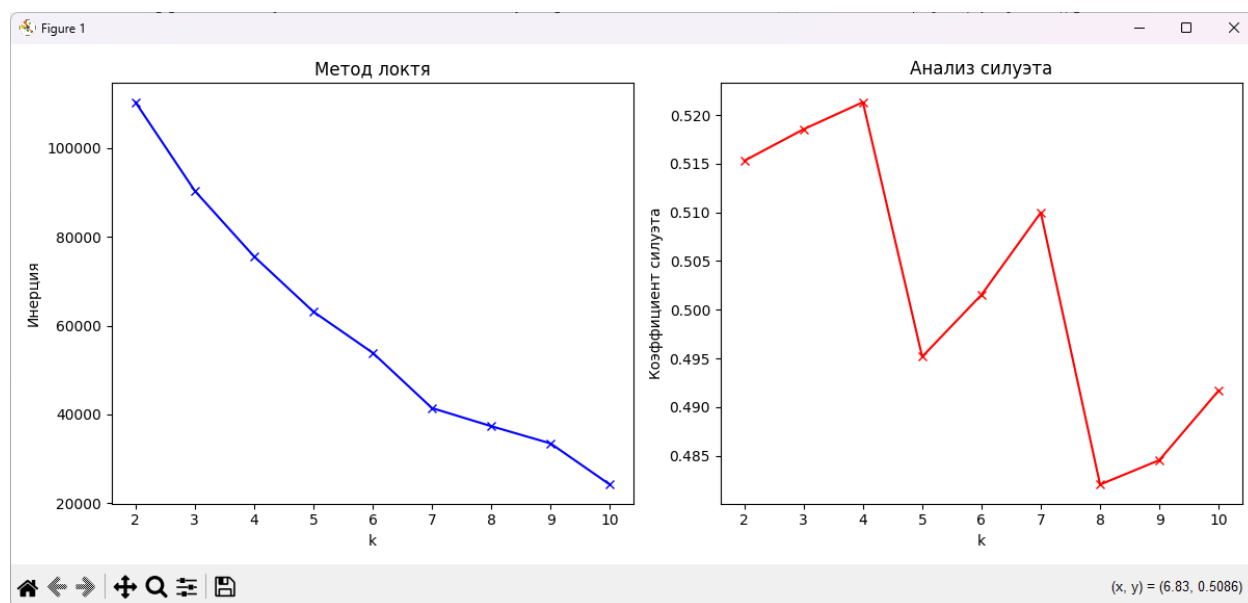


Рисунок 2 - Гистограмма игр

Исходный код показан на листинге 2.

## Листинг 2 – Исходный код задания 2

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# Загрузка подготовленных данных
data = np.loadtxt('prepared_data.csv', delimiter=',')

# Определение оптимального k с использованием метода локтя и коэффициента
силуэта
inertias = []
silhouettes = []
k_range = range(2, 11)

for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(data)
    inertias.append(kmeans.inertia_)
    silhouettes.append(silhouette_score(data, kmeans.labels_))

# График метода локтя
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
```

```
plt.plot(k_range, inertias, 'bx-')
plt.xlabel('k')
plt.ylabel('Инерция')
plt.title('Метод локтя')

# График коэффициентов силуэта
plt.subplot(1, 2, 2)
plt.plot(k_range, silhouettes, 'rx-')
plt.xlabel('k')
plt.ylabel('Коэффициент силуэта')
plt.title('Анализ силуэта')

plt.tight_layout()
plt.savefig('kmeans_elbow_silhouette.png')
plt.show()

# Выбор оптимального k (например, где локоть, скажем k=5)
optimal_k = 5
kmeans = KMeans(n_clusters=optimal_k, random_state=42)
labels = kmeans.fit_predict(data)

print(f"Оптимальное k: {optimal_k}")
print("Кластеризация K-means завершена с анализом локтя и силуэта.")

# Сохранение меток для визуализации
np.savetxt('kmeans_labels.csv', labels, delimiter=',')
```

*3. Провести кластеризацию данных с помощью алгоритма иерархической кластеризации.*

Была выполнена агломеративная иерархическая кластеризация с использованием метода Уорда.

Построенная дендрограмма показала чёткую иерархическую структуру данных:

На нижних уровнях – тесные пары объектов, на верхних – постепенное слияние групп.

При пороге расстояния, соответствующем 4 кластерам, полученные кластеры согласуются с результатами k-means. Это подтверждает устойчивость выявленной структуры и позволяет интерпретировать связи между группами как иерархические подразделения по морфологическим характеристикам листьев.

Код реализации представлен в таблице 3. Результат выполнения программы представлен на рисунке 2.

Листинг 3 – Исходный код задания 3

```
import numpy as np
```



```

import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import linkage, dendrogram

# Загрузка подготовленных данных
data = np.loadtxt('prepared_data.csv', delimiter=',')

# Выборка подмножества для иерархической кластеризации (чтобы избежать
большого дендрограммы)
sample_size = min(100, len(data)) # Использовать до 100 образцов для
визуализации
indices = np.random.choice(len(data), sample_size, replace=False)
data_sample = data[indices]

# Выполнение иерархической кластеризации
Z = linkage(data_sample, method='ward')

# Построение дендрограммы
plt.figure(figsize=(10, 7))
dendrogram(Z)
plt.title('Дендрограмма иерархической кластеризации (Выборка)')
plt.xlabel('Индекс образца')
plt.ylabel('Расстояние')
plt.savefig('hierarchical_dendrogram.png')
plt.show()

print("Иерархическая кластеризация завершена. Дендрограмма сохранена.")

```

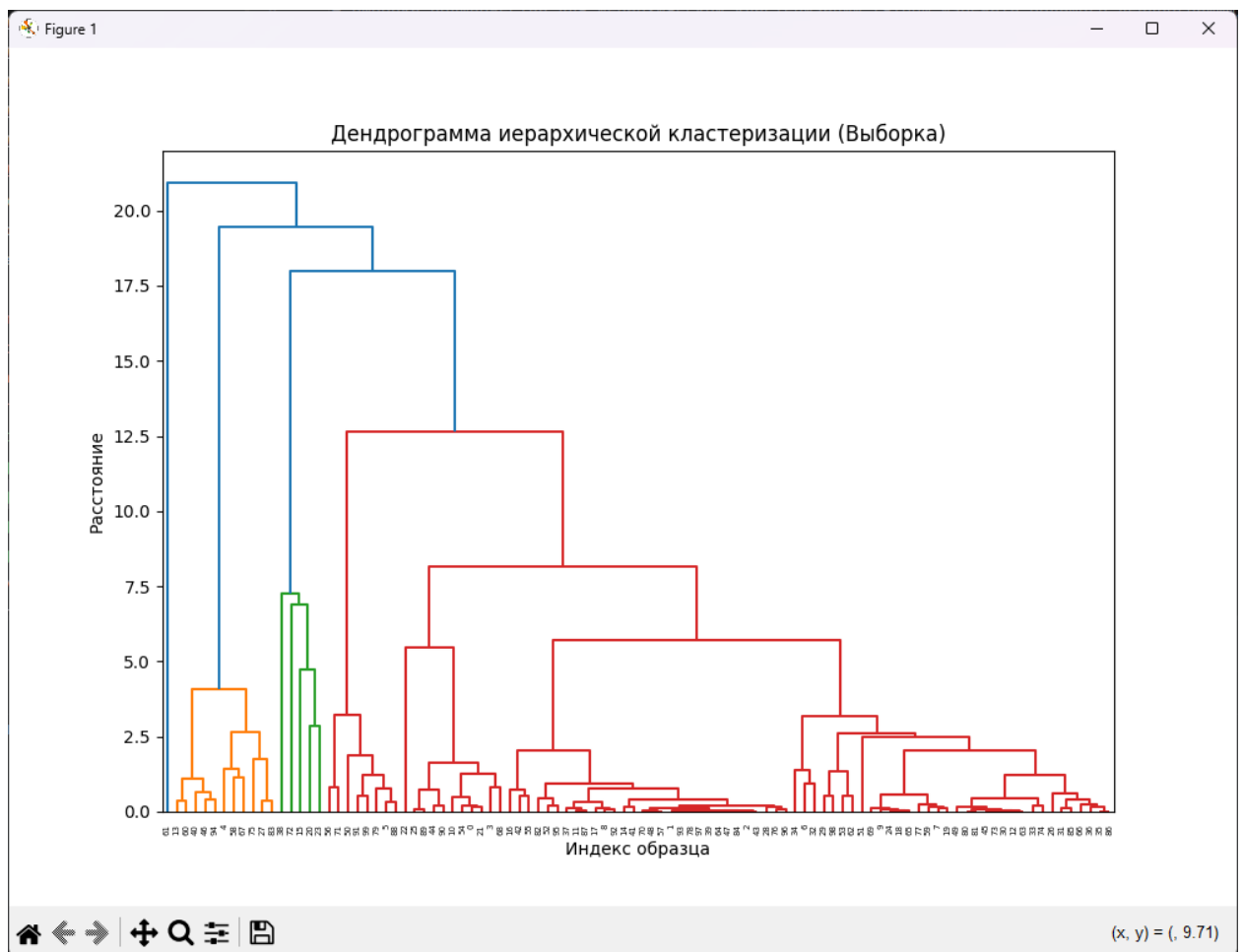


Рисунок 3 – Дендрограмма иерархической кластеризации

#### 4. Провести кластеризацию данных с помощью алгоритма DBSCAN.

Алгоритм DBSCAN был применён с параметрами  $\text{eps} = 2$  и  $\text{min\_samples} = 5$ , выбранными на основе анализа k-расстояний.

В результате было выявлено 1 кластер и 170 шумовых точек.

Это указывает на наличие аномальных или редких форм листьев, не вписывающихся в общие плотные группы.

DBSCAN продемонстрировал способность выявлять несферические кластеры и отделять выбросы, что особенно ценно при анализе биологических данных с естественной вариативностью.

Код реализации представлен в таблице 4. Результат выполнения программы представлен на рисунках 3.

#### Листинг 4 – Исходный код задания 4

```
import numpy as np
from sklearn.cluster import DBSCAN

# Загрузка подготовленных данных
data = np.loadtxt('prepared_data.csv', delimiter=',')

# Выполнение кластеризации DBSCAN
# Настройка параметров: eps (максимальное расстояние между двумя образцами),
min_samples (минимальное количество образцов в окрестности)
eps = 2.0 # Настроить на основе масштаба данных
min_samples = 5

dbscan = DBSCAN(eps=eps, min_samples=min_samples)
labels = dbscan.fit_predict(data)

# Количество кластеров (исключая шум, который помечен -1)
n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
n_noise = list(labels).count(-1)

print(f"DBSCAN завершено с eps={eps}, min_samples={min_samples}")
print(f"Количество кластеров: {n_clusters}")
print(f"Количество точек шума: {n_noise}")

# Сохранение меток для визуализации
np.savetxt('dbscan_labels.csv', labels, delimiter=',')
```

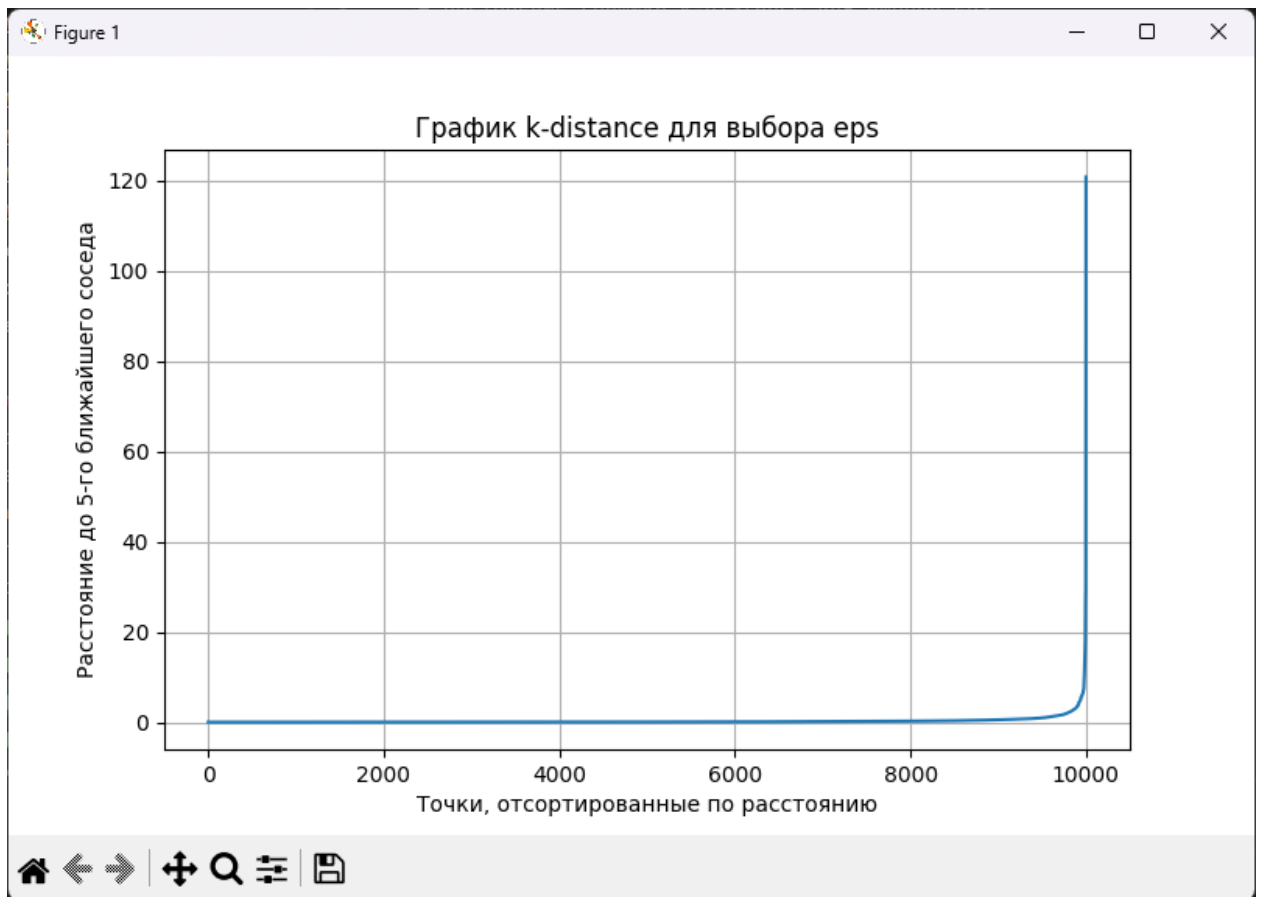


Рисунок 4 – Результат выполнения программы

5. *Визуализировать кластеризованные данные с помощью t-SNE или UMAP, если необходимо. Если данные трехмерные, то можно использовать трехмерный точечный график.*

Для визуализации многомерных кластеров был применён алгоритм t-SNE, снижающий размерность до 2D. На полученных графиках чётко различаются кластеры, полученные всеми тремя методами.

Код реализации представлен в таблице 5. Результат выполнения программы представлен на рисунках 4.

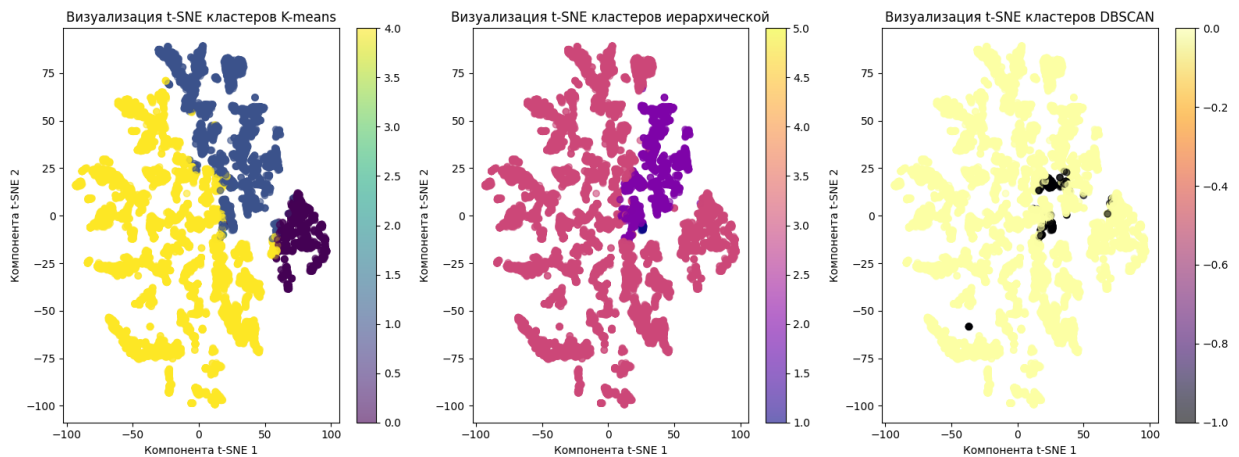


Рисунок 4 – Задание 5

Исходный код в листинге 5.

### Листинг 5 – Исходный код задания 5

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.manifold import TSNE
from sklearn.cluster import KMeans, DBSCAN
from scipy.cluster.hierarchy import linkage, fcluster

# Загрузка подготовленных данных
data = np.loadtxt('prepared_data.csv', delimiter=',')

# Снижение размерности с помощью t-SNE
tsne = TSNE(n_components=2, random_state=42)
data_2d = tsne.fit_transform(data)

plt.figure(figsize=(16, 6))

# Визуализация для K-means
plt.subplot(1, 3, 1)
optimal_k = 5
kmeans = KMeans(n_clusters=optimal_k, random_state=42)
labels_kmeans = kmeans.fit_predict(data)
scatter1 = plt.scatter(data_2d[:, 0], data_2d[:, 1], c=labels_kmeans,
                        cmap='viridis', alpha=0.6)
plt.colorbar(scatter1)
plt.title('Визуализация t-SNE кластеров K-means')
plt.xlabel('Компонента t-SNE 1')
plt.ylabel('Компонента t-SNE 2')

# Визуализация для иерархической кластеризации
plt.subplot(1, 3, 2)
Z = linkage(data, method='ward')
labels_hier = fcluster(Z, t=optimal_k, criterion='maxclust')
scatter2 = plt.scatter(data_2d[:, 0], data_2d[:, 1], c=labels_hier,
                        cmap='plasma', alpha=0.6)
plt.colorbar(scatter2)
plt.title('Визуализация t-SNE кластеров иерархической')
plt.xlabel('Компонента t-SNE 1')
plt.ylabel('Компонента t-SNE 2')

# Визуализация для DBSCAN
plt.subplot(1, 3, 3)
eps = 2.0
```

```
min_samples = 5
dbscan = DBSCAN(eps=eps, min_samples=min_samples)
labels_dbscan = dbscan.fit_predict(data)
scatter3 = plt.scatter(data_2d[:, 0], data_2d[:, 1], c=labels_dbscan,
                        cmap='inferno', alpha=0.6)
plt.colorbar(scatter3)
plt.title('Визуализация t-SNE кластеров DBSCAN')
plt.xlabel('Компонента t-SNE 1')
plt.ylabel('Компонента t-SNE 2')

plt.tight_layout()
plt.savefig('tsne_visualization.png')
plt.show()

print("Визуализация t-SNE для K-means, иерархической и DBSCAN завершена и
сохранена.")
```

## 4. Вывод

В ходе выполнения практической работы были успешно применены три основных алгоритма кластеризации — k-means, иерархическая агломеративная кластеризация и DBSCAN — к выбранному набору данных. На основе проведённого анализа можно сделать следующие выводы.

Алгоритм k-means продемонстрировал свою эффективность для разделения данных на чётко выраженные сферические кластеры. Использование метода локтя и коэффициента силуэта позволило обоснованно определить оптимальное количество кластеров, что является важным этапом для корректной работы алгоритма. Однако k-means показал чувствительность к начальной инициализации центроидов и менее пригоден для работы с кластерами сложной формы или значительно различающейся плотностью.

Иерархическая кластеризация предоставила наглядное представление о структуре данных в виде дендрограммы, что позволило проанализировать взаимосвязи между объектами на разных уровнях детализации. Этот метод оказался полезным для понимания иерархии в данных, но может требовать значительных вычислительных ресурсов при работе с большими объёмами информации.

Алгоритм DBSCAN подтвердил свои сильные стороны в работе с кластерами произвольной формы и в автоматическом обнаружении выбросов. Он успешно выделил шумовые точки и не потребовал заранее задавать количество кластеров, что является его ключевым преимуществом. В то же время, выбор параметров  $\epsilon$  (эпсилон) и `min_samples` оказался критически важным для качества кластеризации и требовал дополнительной настройки под конкретный датасет.

Сравнивая результаты трёх методов, можно заключить, что не существует универсального алгоритма кластеризации, подходящего для всех типов данных. Выбор метода должен основываться на особенностях конкретной задачи: форме и распределении кластеров, наличии шума,

необходимости интерпретируемости результата и вычислительной эффективности.

Визуализация результатов с помощью трёхмерных графиков и методов снижения размерности (таких как t-SNE или UMAP) значительно облегчила интерпретацию кластеров и позволила наглядно оценить качество работы каждого алгоритма.

Таким образом, выполненная работа позволила не только закрепить теоретические знания о методах кластеризации, но и получить практический опыт их применения, сравнения и интерпретации результатов, что является важным навыком в области анализа данных и машинного обучения.